

# Arquitectura e Ingeniería de Computadores

Grado de Ingeniería del Software

## TEMA 6. BUSES

Óscar David Robles Sánchez  
Luis Rincón Córcoles  
David Miraut Andrés  
José Luis Bosque  
Ángel Serrano

## CONTENIDOS

- Introducción a los buses
- Tipos de buses: dispositivos conectados, paralelismo, sincronismo
- Arbitraje del bus
- Ejemplo de estándares de buses: PCI Express

## INTRODUCCIÓN A LOS BUSES

- Un **bus** es un canal de comunicación compartido que utiliza un conjunto de cables para conectar entre sí distintos componentes de un computador.
- Ventajas más importantes de un bus:
  - Versatilidad.
  - Bajo coste.
- Principal desventaja:
  - Cuello de botella en la comunicación.

- Si tuviéramos que conectar cada par de componentes del computador con un cable dedicado, este sistema sería:
  - Muy costoso.
  - Dificilmente escalable para añadir nuevos componentes.
- Los buses permiten conectar todos los componentes del computador mediante un canal de comunicación que comparten todos ellos.
  - Este sistema es versátil porque es fácilmente escalable.
  - Es muy barato.
  - Sin embargo, puede suponer un cuello de botella porque todas las comunicaciones entre componentes se hacen por el mismo bus.
    - El ancho de banda del bus (cuántos bits/s puede transmitir) es un parámetro fundamental del diseño del bus.
    - Además, hay que establecer un protocolo de uso para decidir qué componente escribe en el bus y en qué momento puede hacerlo.

## INTRODUCCIÓN A LOS BUSES

- Velocidad máxima del bus limitada físicamente:
  - Longitud del bus.
  - Número de dispositivos conectados.
- Compromiso entre medidas de rendimiento:
  - Tiempo de respuesta frente a velocidad de transferencia.
- Diseño complejo para soportar diferentes dispositivos (latencias y velocidades de transferencia distintas).

- La velocidad máxima a la que se pueden enviar datos a través del bus depende de:
  - La longitud del bus:
    - Cuanto más largo sea, más lento, ya que las señales eléctricas tardan más en llegar desde el origen al destino de la transmisión.
  - El número de dispositivos conectados:
    - Cuantos más dispositivos, más lento, ya que deben coordinarse más componentes para el proceso de arbitraje (decidir quién puede usar el bus y quién no en un momento dado).
- Hay conflicto de diseño que se resuelve con un compromiso:
  - Para obtener un tiempo de respuesta pequeño se necesita minimizar el tiempo de acceso al bus, para lo cual se debe simplificar el camino de comunicación.
  - Por otra parte, para mantener altas velocidades de E/S, se debe maximizar el ancho de banda del bus.
- El diseño del bus es complicado porque debe permitir la comunicación entre el procesador (muy rápido) con una gran variedad de dispositivos (de latencias y velocidades de transferencia variadas).

## INTRODUCCIÓN A LOS BUSES

### ○ Tipos de líneas en un bus:

- **Líneas de control:** se usan para señalar peticiones y reconocimientos, y para indicar qué tipo de información pasa por las líneas de datos.
- **Líneas de datos:** llevan la información de la fuente al destino.
  - Datos.
  - Órdenes complejas.
  - Direcciones (si el bus es multiplexado).
- **Líneas de dirección:** presentes en algunos buses para permitir comunicar direcciones y datos en una sola transferencia.

- El bus no deja de ser un conjunto de cables (líneas) que transmiten señales eléctricas.
  - Cada línea transmite 1 bit de información.
- Clasificación de las líneas de un bus:
  - Líneas de control:
    - Son las señales generadas por la Unidad de Control y que se envían a todos los componentes del computador para su adecuado funcionamiento en el momento preciso.
      - Señales de petición.
      - Señales de reconocimiento.
      - Señales que activan o desactivan los componentes (multiplexores, memoria, etc.).
    - Al conjunto de estas líneas se las llama bus de control.
    - Tanto el procesador como el resto de los componentes pueden leer y escribir en el bus de control.
  - Líneas de datos:
    - Son las señales que corresponden a los datos que se están enviando.
    - A veces también llevan comandos u órdenes complejas hacia los periféricos.
    - Al conjunto de estas líneas se las llama bus de datos.
    - El ancho del bus de datos suele coincidir con el tamaño de palabra del computador (tamaño de los operandos de la ALU).
    - Tanto el procesador como el resto de los componentes pueden leer y escribir en el bus de datos.
  - Líneas de dirección:
    - Son las señales que transmiten direcciones de memoria.
    - Al conjunto de estas líneas se las llama bus de direcciones.
    - El ancho del bus de direcciones limita el tamaño máximo de la memoria.
    - El procesador es el único que puede escribir en el bus de direcciones.
    - La memoria principal y los periféricos solo pueden leer este bus.
    - Si no hay bus de direcciones independiente, las direcciones se envían usando el bus de datos.
      - En este caso se dice que el bus está multiplexado (envía datos o direcciones según el momento).

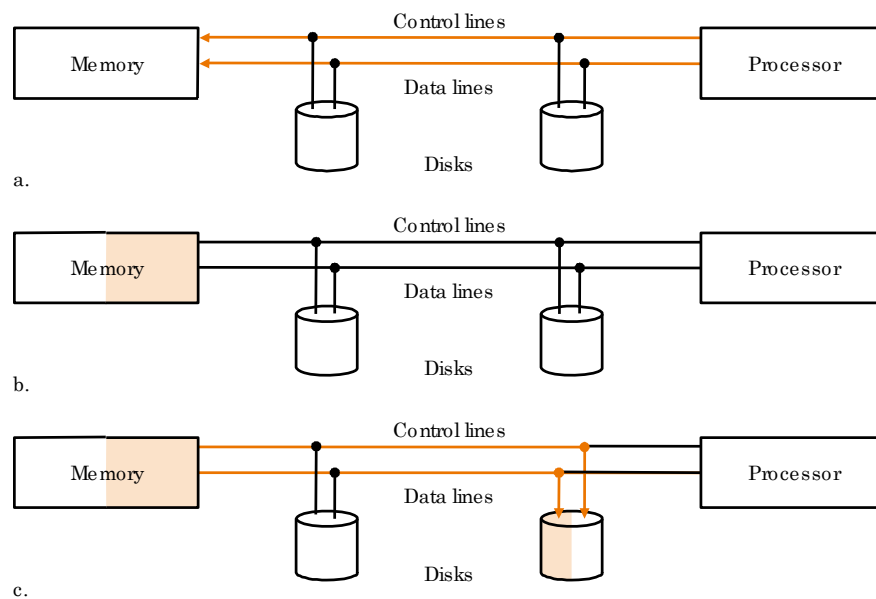
## INTRODUCCIÓN A LOS BUSES

- Transacciones en un bus: reguladas **siempre** por un protocolo de comunicación.
- Una transacción de bus típica tiene dos partes:
  - Envío de la dirección.
  - Envío o recepción de los datos.
- Tipos de transacciones:
  - Entrada: datos viajan del dispositivo a la memoria.
  - Salida: datos viajan de la memoria al dispositivo.

6

- Puesto que el bus es un recurso compartido por todos los componentes del computador, se hace necesario un protocolo de uso.
  - De esta manera, se establece el formato de todas las transacciones del bus.
    - El dispositivo que tiene acceso al bus (maestro) publica una dirección de otro componente:
      - Si es un envío de datos, es la dirección del destino de la transferencia.
      - Si es una petición de datos, es la dirección del componente que tiene los datos y al que le requerimos que los envíe por el bus.
    - A continuación, se produce el envío de datos a través del bus, en el sentido adecuado.
- Las operaciones del bus se clasifican desde el punto de vista del procesador:
  - Entrada:
    - Es cuando el procesador le pide a un dispositivo que envíe datos a la memoria para operar con ellos.
  - Salida:
    - Es cuando el procesador ordena a la memoria que envíe datos hacia otro dispositivo.

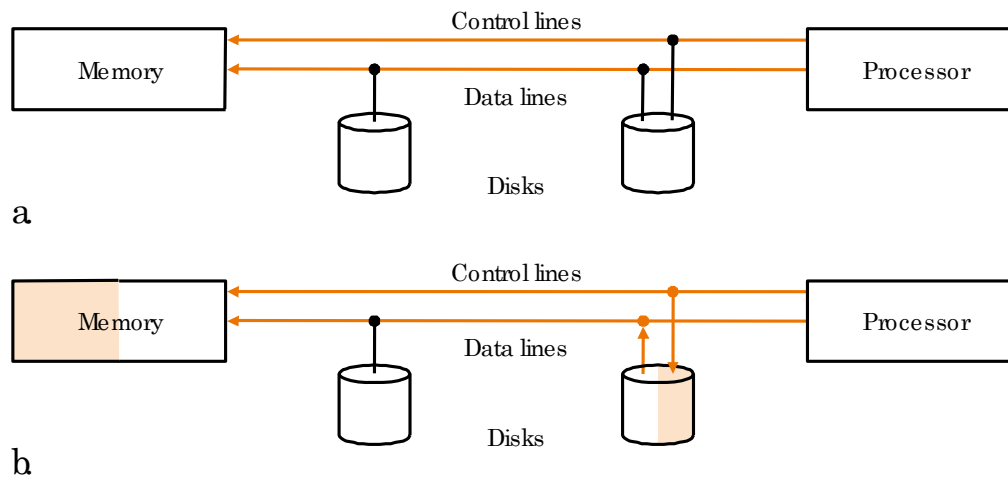
## INTRODUCCIÓN A LOS BUSES



7

- En esta transparencia se esquematiza una operación de salida (envío de datos desde la memoria hacia otro dispositivo):
  - Criterio de colores:
    - Los elementos que participan activamente en la comunicación se muestran en color anaranjado.
    - Se colorea la parte derecha si el dispositivo hace una lectura y la parte izquierda si el dispositivo realiza una escritura.
    - Obsérvese que, en esta figura, las líneas de datos del bus pueden transportar tanto direcciones (es el caso de a) como datos (caso de c).
  - Pasos de la operación:
    - El procesador envía una petición lectura a la memoria por parte del procesador.
      - Las líneas de control señalizan la petición de lectura a memoria, mientras que las líneas de datos contienen la dirección de memoria involucrada.
    - La memoria accede a los datos solicitados (realiza la lectura).
    - La memoria transfiere los datos leídos usando las líneas de datos del bus e indica al dispositivo de E/S que los datos están disponibles, usando las señales de control.
      - El dispositivo almacena los datos tan pronto como aparecen en el bus.

## INTRODUCCIÓN A LOS BUSES



8

- En esta transparencia se esquematiza una operación de entrada (envío de datos desde un dispositivo hacia la memoria):
  - Requiere menos tiempo de actividad porque el dispositivo no tiene que esperar a que la memoria acceda al dato.
  - Pasos de la operación:
    - a) Las líneas de control indican una petición de escritura en memoria, mientras que las líneas de datos contienen la dirección.
    - b) El segundo paso se produce cuando la memoria está lista e informa al dispositivo, que transfiere entonces los datos.
      - Normalmente, la memoria almacenará los datos en cuanto los reciba.
      - El dispositivo no necesita esperar a que acabe la escritura.
  - Se ha supuesto que el dispositivo tiene que esperar a que la memoria le indique que está lista.
    - Esto no es así en algunos sistemas que usan *buffers* o tienen un sistema de memoria rápido.

## CONTENIDOS

- Introducción a los buses
- Tipos de buses: dispositivos conectados, paralelismo, sincronismo
- Arbitraje del bus
- Ejemplo de estándares de buses: PCI Express

## TIPOS DE BUSES

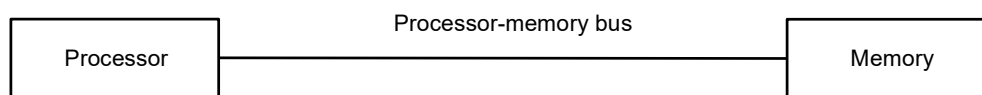
- Según los dispositivos conectados:
  - Bus procesador-memoria
  - Bus *backplane*
  - Bus de E/S
- Según el grado de paralelismo:
  - Bus paralelo
  - Bus serie
- Según el sincronismo:
  - Bus síncrono
  - Bus asíncrono

- Los buses se pueden clasificar según diversos criterios.

## TIPOS DE BUSES: DISPOSITIVOS CONECTADOS

### ○ Bus procesador-memoria:

- Poca longitud.
- Suele ser de alta velocidad.
- Síncrono.
- Adaptado al sistema de memoria para maximizar el ancho de banda entre procesador y memoria.
- Normalmente es un bus propietario.



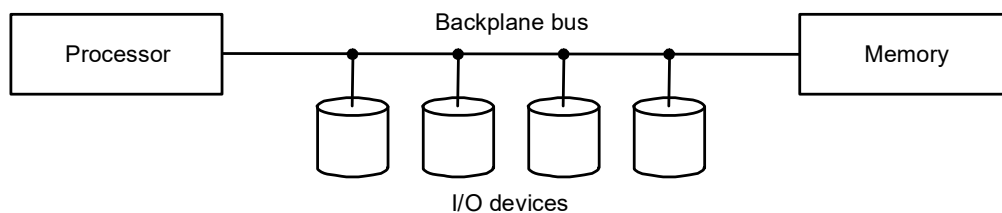
11

- En cuanto a los dispositivos conectados al bus, podemos hablar de:
  - Bus procesador-memoria:
    - Como su nombre indica, es el bus que comunica la memoria principal con el procesador.
    - Como la memoria está cerca del procesador, es un bus de poca longitud.
    - Además, el acceso a la memoria debe ser lo más rápido posible, con lo que es de alta velocidad.
    - El acceso del bus está controlado por una señal de reloj (bus síncrono).
    - Suele ser un bus propietario, es decir, cada fabricante tiene el suyo propio y no publica detalles de su diseño.

## TIPOS DE BUSES: DISPOSITIVOS CONECTADOS

### ○ Bus *backplane*:

- Diseñados para la coexistencia de procesador, memoria y dispositivos de E/S en un único bus.
- Equilibrio entre necesidades de procesador y memoria y las de dispositivos de E/S.



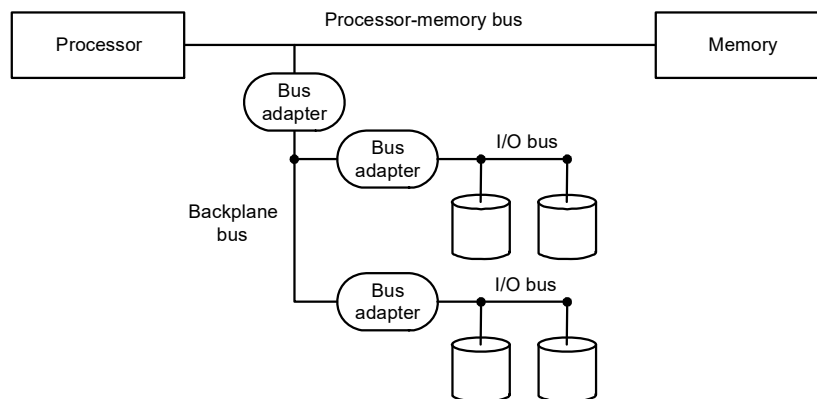
12

- En cuanto a los dispositivos conectados al bus, podemos hablar de:
  - Bus *backplane*:
    - Reciben este nombre porque se construyen en el “plano posterior” del computador, donde se conectan los dispositivos de E/S.
    - Permiten coexistir en un bus al procesador, la memoria y los dispositivos de E/S.
    - Para ello deben equilibrar los requerimientos para la comunicación procesador-memoria y los de la comunicación memoria-dispositivos de E/S.
    - Es un bus asíncrono (las operaciones no dependen de una señal de reloj).
    - A diferencia del bus procesador-memoria (propietario), el *backplane* tiene un diseño estandarizado para permitir la conexión de dispositivos desconocidos.
    - Bajo coste.
    - Ejemplos:
      - ISA.
      - PCI.

## TIPOS DE BUSES: DISPOSITIVOS CONECTADOS

### ○ Bus de E/S:

- Pueden ser de mayor longitud.
- Conexión de diferentes tipos de dispositivos.
- Sin interfaz directa con la memoria.

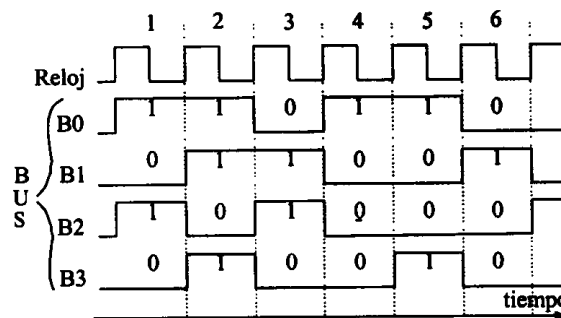


13

- En cuanto a los dispositivos conectados al bus, podemos hablar de:
  - Bus de Entrada/Salida (E/S):
    - Es un bus separado del bus procesador-memoria y se usa solo para operaciones de entrada/salida.
    - Para comunicar los datos entre memoria y dispositivos de E/S, los buses de E/S se conectan al bus procesador-memoria a través de un adaptador de bus, que se encarga de adaptar las velocidades de los buses.
    - En la imagen se muestra:
      - El procesador y la memoria conectados a través de un bus procesador-memoria de alta velocidad.
      - Conectado a dicho bus hay un adaptador del que cuelga un bus *backplane*.
      - A su vez, a través de sendos adaptadores, hay dos buses de entrada salida con diversos dispositivos.
  - Este tipo de organización jerárquica permiten que convivan en el computador componentes con diferentes velocidades.

## TIPOS DE BUSES: PARALELISMO

- **Bus paralelo:** contiene **n** hilos (uno por bit) para transmitir un dato completo en un solo ciclo de reloj.
- Inconveniente: sesgo del reloj influye en altas velocidades.
- Ejemplo: transferencia de 6 datos en un bus de 4 bits.
  - Dato 1: 0101
  - Dato 2: 1011
  - Dato 3: 0110
  - Dato 4: 0001
  - Dato 5: 1001
  - Dato 6: 0010

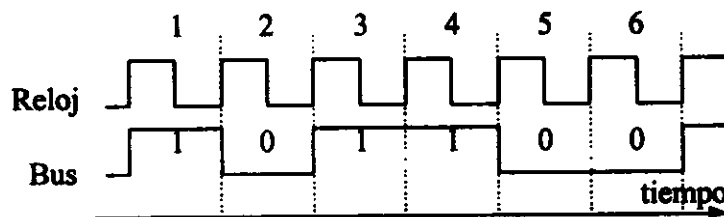


14

- En cuanto al grado de paralelismo del bus, podemos hablar de:
  - Bus paralelo:
    - Permite el envío simultáneo de n bits en el mismo ciclo de reloj.
  - Inconveniente:
    - Si el bus es largo y la frecuencia de reloj es elevada, se produce el llamado sesgo de reloj (*clock skew*).
      - Las señales llegar con retardo a los dispositivos conectados más lejos, con lo que se pierde la capacidad de sincronización.
      - Los buses paralelos compartidos están cayendo en desuso en los computadores actuales.
- En la imagen, el ancho del bus es de 4 bits.
  - De esta forma, se logran enviar 6 datos de 4 bits en un total de 6 ciclos de reloj.

## TIPOS DE BUSES: PARALELISMO

- **Bus serie:** transmite información por un único hilo.
  - Un dato debe transmitirse en varios ciclos de reloj consecutivos.
- Ejemplo: transferencia del dato de 6 bits 101100.



- Bits adicionales de arranque y de parada para sincronizar a emisor y receptor.

15

- En cuanto al grado de paralelismo del bus, podemos hablar de:
  - Bus serie:
    - Envía los bits de uno en uno.
    - En las transferencias serie suele haber bits adicionales de arranque y de parada que no forman parte del dato, pero sirven para sincronizar el emisor y el receptor.
    - Como ya hemos dicho en la transparencia anterior, debido a los problemas de gestión de los buses paralelos compartidos, la industria está derivando hacia buses serie de muy alta velocidad e interconexiones punto a punto.
      - Ejemplo: bus PCI Express.
    - En la imagen se muestra el envío de un dato de 6 bits en un total de 6 ciclos de reloj (no se han dibujado los bits de arranque y parada).

## TIPOS DE BUSES: SINCRONISMO

### Bus síncrono:

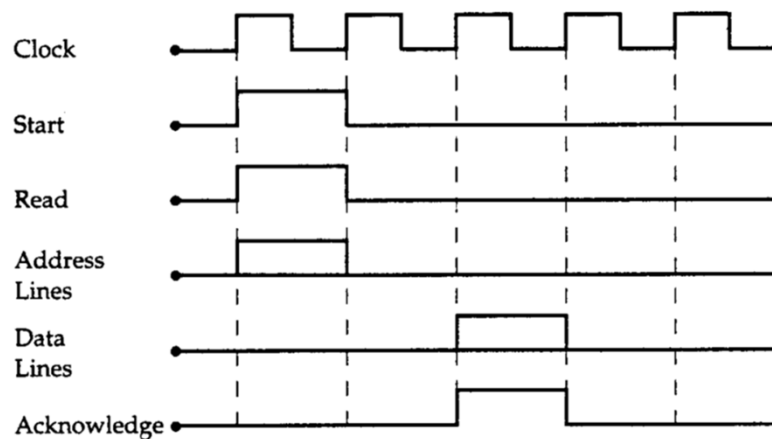
- Tiene una señal de reloj entre las líneas de control.
- Sigue un protocolo para la comunicación.
- La comunicación está gobernada por la señal de reloj.
- Desventajas:
  - Todos los dispositivos conectados a misma velocidad.
  - Sesgo de reloj (*clock skew*) impide conjugar longitud con rapidez.

16

- En cuanto al grado de sincronismo del bus, podemos hablar de:
  - Bus síncrono:
    - Una señal de reloj (CLK) sincroniza todas las operaciones, que deben producirse en el flanco activo del reloj.
    - Los buses procesador-memoria suelen ser síncronos porque:
      - Los dispositivos involucrados están cerca.
      - Son pocos dispositivos, que trabajan todos a la misma velocidad.
      - Están preparados para trabajar a velocidades de reloj altas.
      - Sesgo del reloj (*clock skew*): un bus síncrono no puede ser a la vez largo y rápido.

## TIPOS DE BUSES: SINCRONISMO

- Ejemplo de funcionamiento de un bus síncrono: lectura de memoria en un bus procesador – memoria.



17

- En cuanto al grado de sincronismo del bus, podemos hablar de:
  - Bus síncrono:
    - Ejemplo de uso:
      - Para la realización de una lectura de memoria en un bus procesador-memoria, se podría tener un protocolo que transmita la dirección y la orden de lectura en el primer ciclo de reloj, usando las líneas de control para indicar el tipo de petición.
      - La memoria entonces debería suministrar los datos en un determinado ciclo de reloj.
      - Este tipo de protocolo se puede implementar fácilmente con un pequeño autómata. Dado que el protocolo está predeterminado el bus podrá funcionar a gran velocidad.

## TIPOS DE BUSES: SINCRONISMO

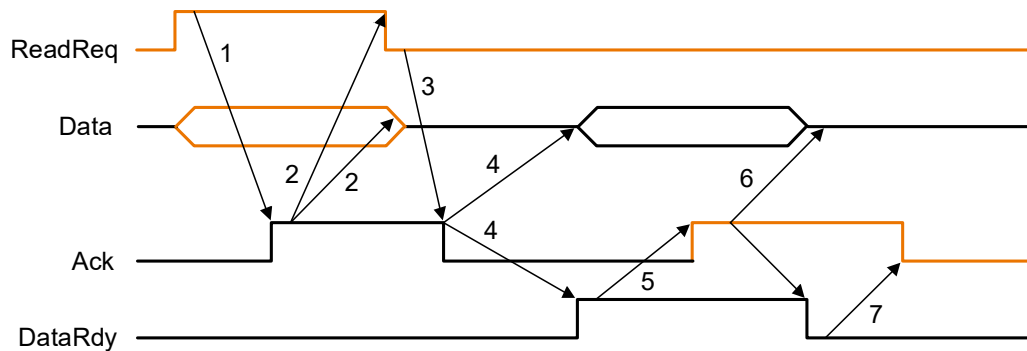
### Bus asíncrono:

- Carece de reloj.
- Puede admitir una variedad más amplia de dispositivos.
- Sin problemas de longitud por sincronización o sesgo de reloj.
- Protocolo de apretón de manos (*handshaking*).
- Líneas adicionales de control:
  - *Read Request (ReadReq)*
  - *Data Ready (DataRdy)*
  - *Acknowledge (Ack)*

18

- En cuanto al grado de sincronismo del bus, podemos hablar de:
  - Bus asíncrono:
    - No necesitan reloj.
    - Permite trabajar con dispositivos diversos (con distintas velocidades).
    - Usan un protocolo de apretón de manos:
      - Se siguen una serie de pasos, de forma que el emisor y el receptor solo avanzan al paso siguiente cuando ambos están de acuerdo.
      - Este protocolo necesita señales de control adicionales:
        - *Read Request (ReadReq)*:
          - Se activa cuando se produce una petición de lectura de memoria.
          - Simultáneamente la dirección se pone en las líneas de datos.
        - *Data Ready (DataRdy)*:
          - Se activa cuando la palabra de datos ya está disponible en las líneas de datos.
        - *Acknowledge (Ack)*:
          - Se activa cuando se ha reconocido la activación de cualquiera de las dos señales anteriores.
          - En ese instante se desactiva la señal anterior.

## EJEMPLO DE *HANDSHAKING*



19

Petición de lectura de un dato que realiza un **dispositivo de E/S** a la **memoria**

- Ejemplo de protocolo de *handshaking* paso a paso:
  - Se muestra el caso de una petición de lectura de un dato que realiza un dispositivo de E/S (en naranja) a la memoria (en negro).
  - En este ejemplo se usa un bus multiplexado (a veces transporta datos y otras, direcciones).
  - Las flechas indican relaciones de causa-efecto.
  - El dispositivo de E/S activa la señal *ReadReq* y coloca una dirección en el bus. A partir de este momento:
    1. Cuando la memoria detecta la señal *ReadReq*, lee la dirección que hay en el bus y activa *Ack* para indicar que ha detectado la petición.
    2. El dispositivo de E/S detecta que *Ack* ha sido activada. Entonces desactiva *ReadReq* y libera el bus.
    3. La memoria detecta que *ReadReq* ha sido desactivada y entonces desactiva la señal *Ack* para dar por terminado el reconocimiento de la petición. Durante un periodo equivalente al tiempo de acceso a la memoria se busca el dato solicitado.
    4. Este paso se inicia cuando la memoria tiene el dato preparado. Pone el dato en el bus y activa *DataRdy*.
    5. El dispositivo de E/S detecta que *DataRdy* está activa, lee los datos del bus e indica, activando *Ack*, que ya tiene los datos.
    6. La memoria detecta la señal *Ack*, desactiva *DataRdy* y libera el bus.
    7. Finalmente, el dispositivo de E/S, al detectar que *DataRdy* se desactiva, a su vez desactiva la señal *Ack*, indicando de esta forma que la transmisión ha terminado.

## CONTENIDOS

- Introducción a los buses
- Tipos de buses: dispositivos conectados, paralelismo, sincronismo
- Arbitraje del bus
- Ejemplo de estándares de buses: PCI Express

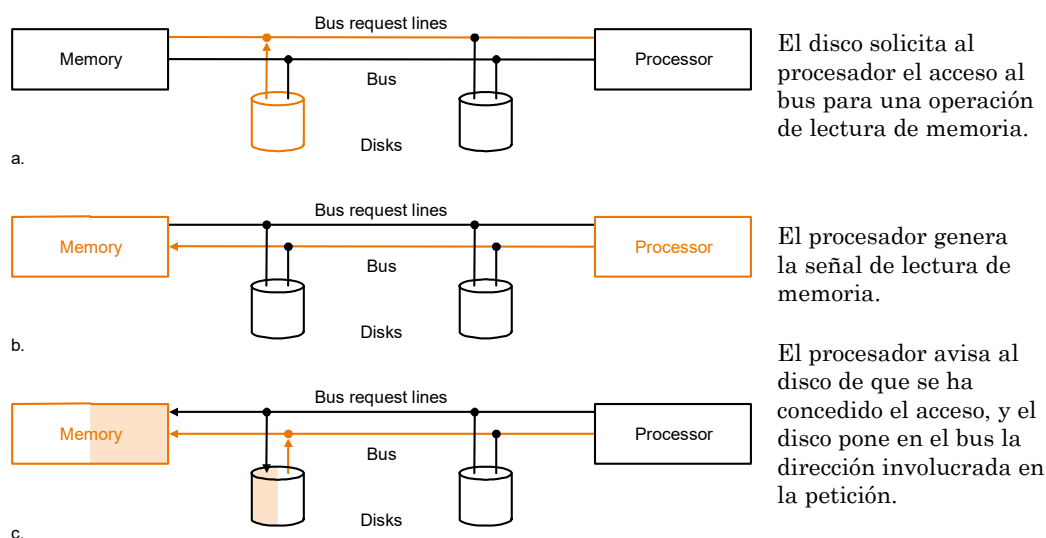
## ARBITRAJE DEL BUS

- Un **maestro** del bus iniciará y controlará toda petición de bus.
- El procesador siempre es un maestro del bus.
- La memoria nunca lo es.
- Sistema más sencillo: el procesador es el único maestro del bus.
- Todas las peticiones deberán estar controladas por el procesador.

- Para decidir quién puede tomar el control del bus para realizar una transferencia se debe usar un protocolo de arbitraje.
  - Hay que evitar cargar el procesador con toda operación que tenga que ver con la E/S.
  - Se llama **maestro** del bus al dispositivo que toma el control del bus y realizará toda la operación.
  - El dispositivo al que se requiere un dato será el **esclavo**.
    - El procesador debe ser capaz de iniciar siempre las peticiones de bus para acceder a memoria, con lo que siempre es un maestro del bus.
    - La memoria nunca genera peticiones propias, sino que responde a las peticiones de lecturas y escrituras, por lo que es un esclavo del bus.

## ARBITRAJE DEL BUS: MAESTRO ÚNICO

- Ejemplo: pasos iniciales en una transacción de salida de memoria a disco en un bus con un solo maestro.



- Pasos en una operación de E/S con un solo maestro, en la que el procesador (maestro) solicita que se realice una lectura en memoria y que el dato se envíe a un disco:
  - Código de colores:
    - Las líneas y unidades activas en cada paso se muestran en color.
    - El color también indica la fuente de una lectura (memoria) o el destino de una escritura (el disco).
  - Pasos:
    - Primero, el dispositivo genera una petición de bus para indicar al procesador que el dispositivo desea usar el bus.
    - El procesador responde y genera las señales de control del bus adecuadas.
      - Por ejemplo, si el dispositivo desea realizar una salida desde memoria, el procesador activa la señal de petición de lectura de memoria.
    - El procesador también notifica al dispositivo que su petición de bus está siendo procesada; como resultado, el dispositivo sabe que puede usar el bus y pone en él la dirección involucrada en la petición.
  - Gran desventaja de este esquema:
    - El procesador está implicado en todas las transacciones de bus y pierde tiempo que no puede dedicar a ejecutar instrucciones.
    - Como los dispositivos son cada vez más rápidos, involucrar así al procesador no es adecuado.

## ARBITRAJE DEL BUS: MÚLTIPLES MAESTROS

- Esquema alternativo: varios maestros de bus. Se necesita un protocolo de uso (arbitraje).
- Señales específicas:
  - Petición de bus (*bus request*).
  - Concesión de bus (*bus grant*).
  - Liberación de bus (*bus release*).
- Factores en la elección:
  - Prioridad de bus.
  - Imparcialidad (*fairness*).
- Tiempo de arbitraje.

23

- Para evitar el cuello de botella que implica que el procesador sea el maestro de todas las operaciones de E/S, se establece la estrategia de múltiples maestros.
  - Cada maestro tendrá capacidad para iniciar una transferencia.
  - Se necesita un protocolo o mecanismo que indique quién es el siguiente dispositivo con acceso al bus: arbitraje.
    - Se trata de decidir quién es el siguiente maestro que podrá utilizar el bus.
    - Ahora el árbitro no es el procesador.
  - Se necesitan algunas señales específicas para permitir este arbitraje:
    - El dispositivo que desee usar el bus debe señalar una petición de bus (*bus request*) para ser el maestro.
      - También puede ser el procesador.
    - Cuando el árbitro le concede el bus, se lo notifica al peticionario (*bus grant*).
      - Tras detectar esta concesión, el dispositivo puede usar el bus para realizar la operación deseada.
    - Al terminar la operación, debe indicar al árbitro que ya no lo necesita (*bus release*).
      - En este momento el árbitro podrá conceder el bus a otro dispositivo.
  - Factores que influyen en la elección del dispositivo al que se concede el bus:
    - Prioridad de bus:
      - Cada dispositivo tiene una prioridad.
      - Los dispositivos más prioritarios deben atenderse antes que los menos prioritarios.
    - Imparcialidad (*fairness*):
      - Ningún dispositivo debe ser ignorado.
      - Todo el que quiera acceder tiene la garantía de que recibirá el bus en algún momento.
        - Quizá si el dispositivo es poco prioritario le tocará esperar más, pero hay que concederle el uso del bus tarde o temprano.
  - El tiempo de arbitraje es una sobrecarga que aumenta el tiempo de acceso al bus.
    - Debe solaparse con transferencias siempre que sea posible.

## ARBITRAJE DEL BUS

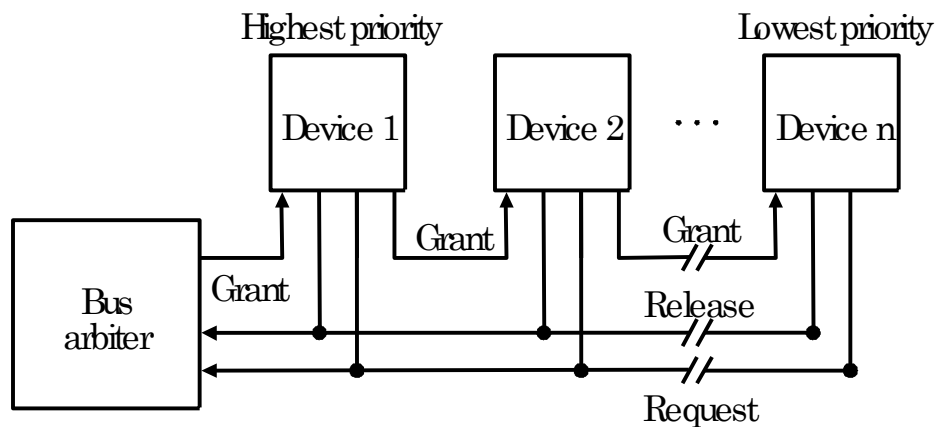
Los esquemas de arbitraje de bus se pueden clasificar en cuatro clases:

- Arbitraje en serie (*daisy chain arbitration*).
- Arbitraje paralelo centralizado.
- Arbitraje distribuido por autoselección.
- Arbitraje distribuido por detección de colisión.

- Existe una amplia variedad de esquemas de arbitraje, que pueden requerir circuitería especial o protocolos extremadamente sofisticados.
  - Estudiaremos los siguientes:
    - En serie o *daisy chain*.
    - Paralelo centralizado.
    - Distribuido por autoselección.
    - Distribuido por detección de colisión.

## ARBITRAJE DEL BUS: EN SERIE

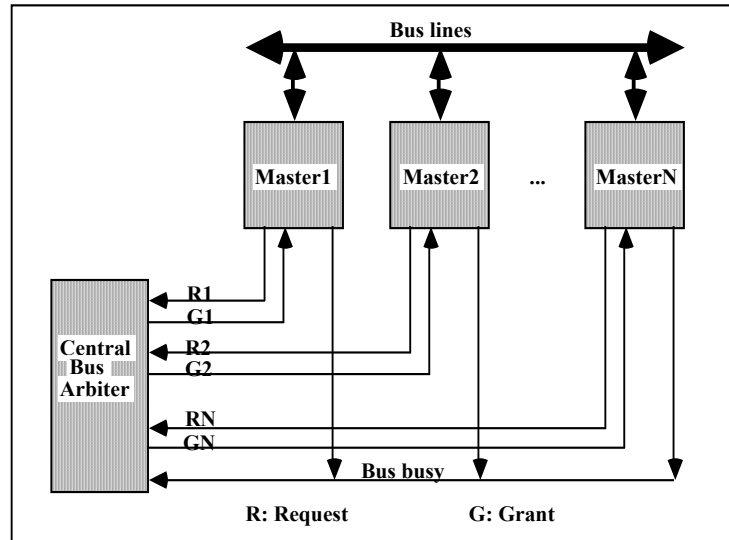
- Un bus *daisy chain* usa una señal de concesión de bus que encadena todos los dispositivos, de más a menos prioritario.



25

- Esquemas de arbitraje de bus:
  - Arbitraje en serie o *daisy chain*:
    - Tenemos varios dispositivos conectados en serie.
    - La prioridad de cada dispositivo viene determinado por su posición en la serie.
      - Cuanto más cerca del árbitro, mayor es la prioridad.
    - Todos los dispositivos envían su señal de petición (*Request*) directamente al árbitro.
      - Nótese que la señal de *Request* consta de un único bit.
        - Vale 1 en cuanto hay al menos 1 dispositivo que quiere usar el bus.
      - Sin embargo, el árbitro no sabe la identidad del dispositivo que pide usar el bus.
    - Sin embargo, la señal de concesión (*Grant*) va pasando de uno en uno.
      - Cuando un dispositivo recibe la señal de concesión pueden pasar dos cosas:
        - Si no tiene activada la señal de petición, es que no quería usar el bus, así que propaga la señal de concesión al siguiente dispositivo de menor prioridad.
        - Si tiene activada la señal de petición, es que quiere usar el bus, así que se entiende que le han concedido su uso y no propaga la señal de concesión al siguiente dispositivo.
      - De esta forma, los dispositivos de alta prioridad interceptan la señal de concesión de bus y no permiten que la vean los menos prioritarios.
        - Este sistema no garantiza la imparcialidad, ya que los dispositivos menos prioritarios (los más alejados del árbitro) pueden que no consigan nunca acceso al bus.
      - La utilización de la señal de concesión de esta forma limita la velocidad del bus.
    - Cuando un dispositivo ha terminado de usar el bus, activa su señal de liberación (*Release*).
      - Al igual que pasa con *Request*, se trata de una señal de un único bit.
        - Vale 1 en cuanto hay al menos 1 dispositivo que ha activado su señal particular de *Release*.

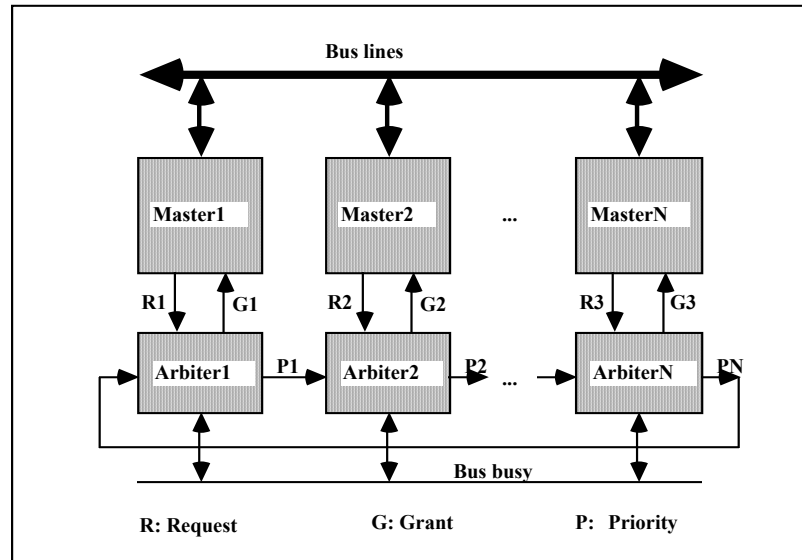
## ARBITRAJE DEL BUS: PARALELO CENTRALIZADO



26

- Esquemas de arbitraje de bus:
  - Arbitraje paralelo centralizado:
    - Cada dispositivo tiene su propia señal de petición (*Request*) para comunicarse con el árbitro.
      - Es decir, los dispositivos piden acceso al bus de forma independiente.
    - Un único árbitro centraliza todas las peticiones y selecciona un dispositivo de entre los que solicitan acceso y notifica al elegido.
      - Es decir, cada dispositivo tiene también su propia señal de concesión (*Grant*).
    - Para informar al árbitro de que están usando el bus, existe una señal de bus ocupado (*Bus busy*).
  - Cuello de botella en el uso del bus.
  - El bus PCI (*backplane* estándar) funciona de esta forma.

## ARBITRAJE DEL BUS: DISTRIBUIDO POR AUTOSELECCIÓN



27

- Esquemas de arbitraje de bus:
  - Arbitraje distribuido por autoselección:
    - En este caso, no se necesita un árbitro centralizado, ya que cada dispositivo determina de forma independiente si es el solicitante de mayor prioridad.
    - Este esquema, sin embargo, requiere más líneas para señales de petición.
    - Procedimiento:
      - Cuando un dispositivo quiere acceder al bus, pone un código que le identifica.
      - El valor contenido en el bus depende de cuántos dispositivos están colocando su código en el bus.
        - Como resultado de esta escritura múltiple, el bus contiene el código del dispositivo más prioritario de entre los que han solicitado el acceso.
        - De esta forma se decide quién usará el bus sin necesidad de un árbitro central.
    - Ejemplo:
      - NuBus ("new bus").
        - Bus *backplane* de 32 bits usado en el Macintosh II de Apple (años 80 del siglo XX).
        - Como todos los dispositivos conectados al bus conocían su código identificativo, no hacía falta configurarlos.
        - De esta forma, fue un de los primeros ejemplos de arquitectura "*plug-and-play*".

## ARBITRAJE DEL BUS: DISTRIBUIDO POR DETECCIÓN DE COLISIÓN

- Los dispositivos solicitan el bus de forma independiente.
- En caso de 2 o más peticiones simultáneas: colisión.
- Se selecciona un dispositivo de entre los que colisionan.
- Ethernet.

28

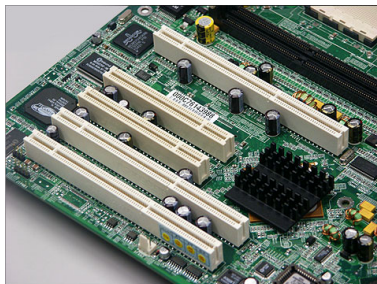
- Esquemas de arbitraje de bus:
  - Arbitraje distribuido por detección de colisión:
    - En este caso, los dispositivos solicitan el bus de forma independiente por una línea común.
    - Se llama “colisión” cuando dos o más dispositivos quieren usar el bus en el mismo instante.
    - Cuando un dispositivo quiere transmitir por el bus, comprueba si el bus está ocupado mediante la lectura de su contenido:
      - Si escribe un dato en el bus y lee lo mismo que ha escrito, es que el bus estaba libre.
        - En ese caso, continúa con la transmisión hasta que la termine o hasta que se detecte una colisión.
      - Si, por el contrario, escribe un dato en el bus y lee algo diferente, es que el bus está ocupado y hay otro dispositivo usándolo.
        - Es cuando se produce la colisión.
        - Cuando un dispositivo detecta una colisión, detiene su transmisión y se inhibe (espera) durante un tiempo antes de volver a intentarlo.
          - El tiempo es aleatorio (y, por tanto, diferente para cada dispositivo), de tal forma que uno de los dispositivos es el que esperará menos y podrá reanudar (o iniciar) su transmisión sin peligro de colisión.
    - El número de dispositivos en el bus está limitado para evitar un exceso de colisiones.
    - Ejemplo de uso:
      - Ethernet.
        - Allí en concreto este algoritmo se llama *Carrier-sense multiple access with collision detection* (CSMA/CD).

## CONTENIDOS

- Introducción a los buses
- Tipos de buses: dispositivos conectados, paralelismo, sincronismo
- Buses síncronos y asíncronos
- Ejemplo de estándares de buses: PCI Express

## BUS PCI

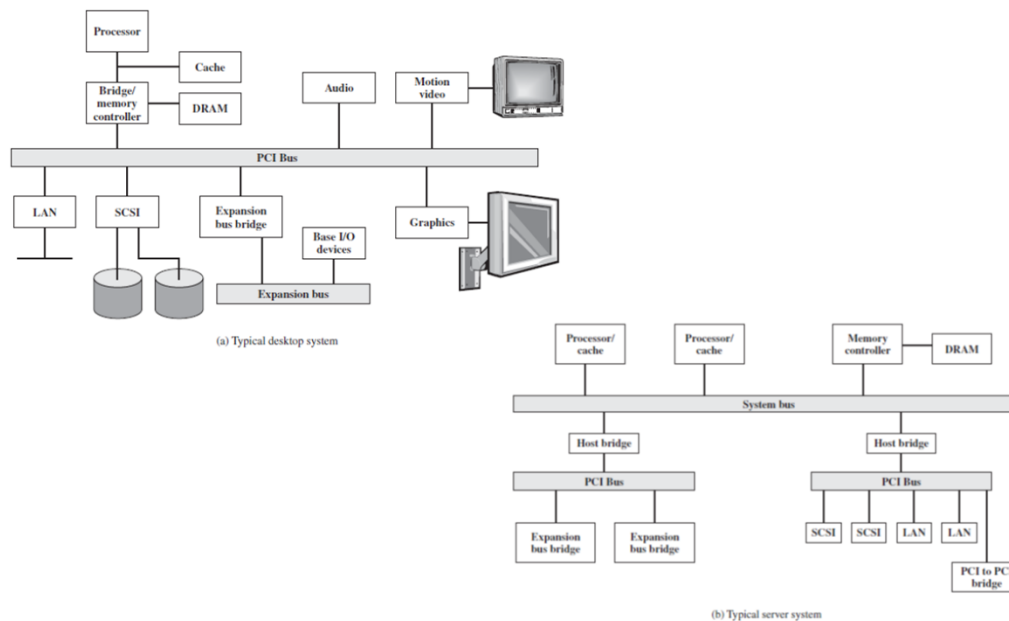
- PCI = *Peripheral Component Interconnect*.
- Ancho de banda elevado e independiente del procesador.
- Puede configurarse como un bus de 32 o 64 bits. Cuando es para 64 bits, admite una frecuencia de 66 MHz, lo que equivale a una velocidad de transferencia de  $528 \text{ MB/s} = 4,224 \text{ Gb/s}$ .



30

- Antes de estudiar el bus PCI Express, necesitamos conocer su antecesor directo: PCI.
  - PCI = *Peripheral Component Interconnect*, interconector de componentes periféricos.
  - A veces llamado también PCI convencional para distinguirlo de otras variantes, como:
    - PCI Extended (PCI-X).
    - PCI Express (PCIe).
      - Este último viene sustituyendo a PCI desde 2003.
  - Se trata de un bus de tipo *backplane*, por tanto, es un bus paralelo.
  - Fue creado en 1990 por Intel para sus procesadores Pentium.
    - Rápidamente liberó las patentes al dominio público y promovió la asociación industrial PCI SGI (Grupo de Interés Especial de PCI, por sus siglas en inglés).
    - Esto favoreció su adopción en todo tipo de sistemas informáticos de diferentes fabricantes.
  - Trabaja tanto por transferencias simples como por ráfagas.
  - Es de tipo multiplexado, lo que quiere decir que se usan los mismos hilos para direcciones y para datos.
  - Los ciclos de bus tienen siempre una fase inicial en la que un maestro adquiere acceso al bus.
  - Después se especifica el tipo de transacción, que puede ser:
    - Aceptación de interrupción.
    - Ciclo especial.
    - Lectura E/S.
    - Escritura E/S.
    - Lectura de memoria.
    - Lectura de línea de memoria.
    - Lectura múltiple de memoria.
    - Escritura en memoria.
    - Escritura en memoria con invalidación.
    - Lectura de configuración.
    - Escritura de configuración.
    - Ciclo de doble dirección.
  - Velocidad de transferencia en el caso del bus PCI de 64 bits:
    - $8 \text{ bytes} * 66 \text{ MHz} = 528 \text{ MB/s}$  ( $1 \text{ MB/s} = 10^6 \text{ B/s}$ ).
    - $1 \text{ Gb/s} = 10^3 \text{ Mb/s}$ .
  - Imagen izquierda:
    - Pueden verse varias ranuras (*slots*) para tarjetas de expansión que usan el bus PCI (en color blanco).
      - Las dos centrales son PCI de 32 bits de 5 V.
      - Las otras tres, de mayor tamaño, son PCI de 64 bits de 3,3 V.
  - Imagen derecha:
    - Ejemplo de tarjeta de expansión para un bus PCI de 32 bits de 5 V.
    - En este caso, se trata de una tarjeta de ethernet para conexión a Internet.

## BUS PCI



31

- En esta diapositiva se muestran dos posibles ejemplos de configuraciones del bus PCI:
  - Típico ordenador de escritorio.
    - Existe un único bus PCI al que se conectan diferentes dispositivos, como:
      - Tarjeta de red (LAN).
      - Discos duros (SCSI).
      - Tarjeta de audio.
      - Tarjeta gráfica.
      - Tarjeta de vídeo.
      - El procesador y la memoria están conectados entre sí de manera independiente al bus PCI.
      - Puede haber un bus de expansión conectado mediante un puente para otros dispositivos.
  - Típico servidor.
    - Ahora disponemos de varios procesadores (cada uno con su caché) y la memoria conectados a través del bus del sistema.
    - Este sistema dispone de dos buses PCI separados, cada uno con sendos puentes que los conectan a dicho bus del sistema.
    - De cada bus PCI cuelgan diferentes dispositivos.

Imágenes tomadas de Stallings, Computer Organization and Design, Prentice Hall, 8.ª edición, 2010, pág. 96.

## BUS PCI EXPRESS

- Desde 2003, sustituye a PCI, PCI-X y AGP (este último en tarjetas gráficas).
- Interconexión punto a punto (serie), nuevo concepto que sustituye al de bus paralelo de uso compartido.
- Independiente del procesador.
- Alto ancho de banda para permitir operaciones pesadas de E/S.
- Capaz de gestionar diferentes transferencias a la vez, dando prioridad a los datos más urgentes.

32

- Bus PCI Express (PCIe):
  - Surgió porque el bus PCI no pudo mantener el ritmo creciente de velocidad de transferencia que necesitaban los periféricos.
    - Tampoco el PCI-X ni el AGP (usado en las tarjetas gráficas) pudieron evolucionar tan rápido como se necesitaba.
  - Con PCIe se abandona la configuración de bus paralelo compartido de PCI para convertirse en un esquema de interconexión punto a punto, por tanto, con comunicación serie.
    - Conecta cada par de componentes, sin ser un bus compartido entre múltiples dispositivos.
  - Es un bus que puede usarse con cualquier procesador, lo cual lo hace muy versátil.
  - Permite un gran ancho de banda para envíos masivos de datos, como pueden ser los vídeos bajo demanda en tiempo real (*streaming*).
    - Permite la realización de múltiples transferencias concurrentes mediante un sistema de prioridades según el tipo de transmisión.
      - Es decir, los datos de mayor prioridad se envían primero.
    - Las prioridades se indican mediante etiquetas que se adjuntan a los datos enviados.
      - Por ejemplo, es muy importante procesar los datos de *streaming* primero, ya que los datos de tiempo real que llegan tarde no sirven para nada.

## BUS PCI EXPRESS

| Specifications          | Lanes    |         |          |          |          |
|-------------------------|----------|---------|----------|----------|----------|
|                         | x1       | x2      | x4       | x8       | x16      |
| 2.5 GT/s (PCIe 1.x +)   | 500 MB/S | 1 GB/S  | 2 GB/S   | 4 GB/S   | 8 GB/S   |
| 5.0 GT/s (PCIe 2.x +)   | 1 GB/S   | 2 GB/S  | 4 GB/S   | 8 GB/S   | 16 GB/S  |
| 8.0 GT/s (PCIe 3.x +)   | 2 GB/S   | 4 GB/S  | 8 GB/S   | 16 GB/S  | 32 GB/S  |
| 16.0 GT/s (PCIe 4.x +)  | 4 GB/S   | 8 GB/S  | 16 GB/S  | 32 GB/S  | 64 GB/S  |
| 32.0 GT/s (PCIe 5.x +)  | 8 GB/S   | 16 GB/S | 32 GB/S  | 64 GB/S  | 128 GB/S |
| 64.0 GT/s (PCIe 6.x +)  | 16 GB/S  | 32 GB/S | 64 GB/S  | 128 GB/S | 256 GB/S |
| 128.0 GT/s (PCIe 7.x +) | 32 GB/S  | 64 GB/S | 128 GB/S | 256 GB/S | 512 GB/S |

Velocidades de transferencia con envío de datos bidireccional

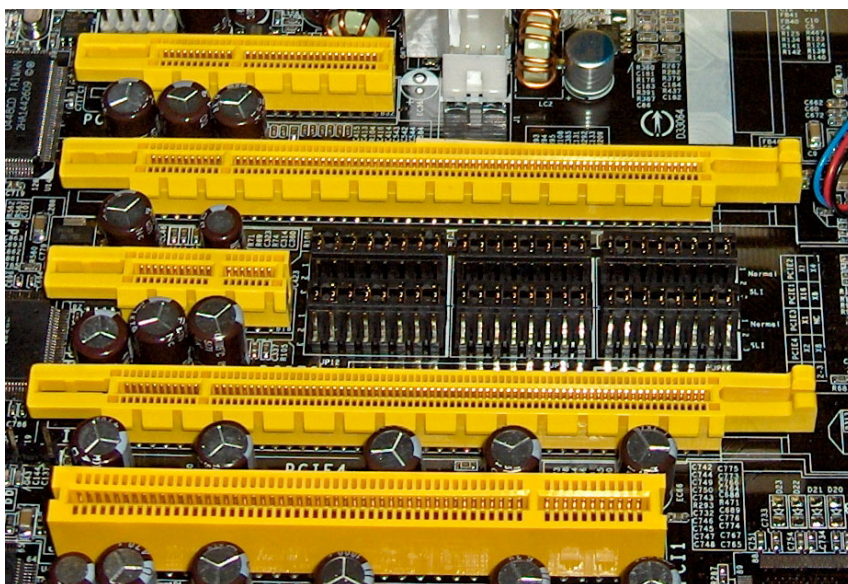
33

- Hasta la fecha existen 7 generaciones de PCIe (la 8.0 está prevista para el año 2028):
  - Cada generación dobla la capacidad de transferencia de la anterior.
  - Entre la primera generación y la séptima hay un factor 64x en velocidad de transferencia, a igualdad de carriles.
  - A igualdad de generación, el formato 16x (16 carriles) supone una velocidad de transferencia 16 veces superior al formato 1x (1 carril).
- En esta transparencia la unidad GT/s se refiere a gigatransferencias por segundo, donde cada transferencia se refiere al envío serie de 1 bit.
  - Como se envían datos codificados durante la transmisión, las velocidades de transferencia expresadas en MB/s o GB/s que aparecen en las casillas de colores están calculadas sobre los datos netos.
- Los valores representados muestran la velocidad de transferencia total, considerando tanto al transmisor (TX) como al receptor (RX) en una comunicación bidireccional. Por eso, podréis encontrar tablas similares en Internet con los valores divididos por la mitad (se refieren solo al emisor).
  - Los valores de la tabla de las dos primeras generaciones son exactos, pero, a partir de 3.x se han redondeado por simplicidad. Los valores en esos casos no son exactos por el tipo de codificación que se aplica a los datos (por ejemplo, la codificación 128b/130b, ver más adelante).

Fuente de la imagen: PCI-SIG, [https://pcsig.com/sites/default/files/files/PCI-SIG\\_Speeds%20and%20Feeds%20Graphic\\_Final.pdf](https://pcsig.com/sites/default/files/files/PCI-SIG_Speeds%20and%20Feeds%20Graphic_Final.pdf)

Más info: <https://pcsig.com/pcie-70-specification-now-available-pci-sig-members>

## BUS PCI EXPRESS

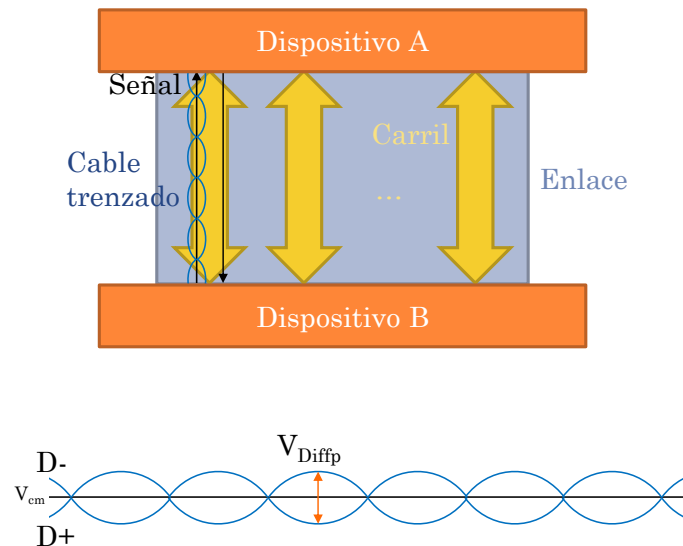


34

- En esta imagen se muestran varias ranuras (*slots*) de PCIe:
  - Los *slots* de PCI Express tienen diferentes versiones: 1x, 2x, 4x, 8x y 16x.
  - De arriba abajo, los *slots* de la imagen son:
    - PCI Express  $\times 4$ .
    - PCI Express  $\times 16$ .
    - PCI Express  $\times 1$ .
    - PCI Express  $\times 16$ .
    - PCI convencional (32 bits, 5 V).
  - Gracias al formato de las ranuras, se puede conectar una tarjeta de un dispositivo  $\times 1$  en una ranura de mayor tamaño (por ejemplo  $\times 16$ ) y funcionaría como si fuera una ranura  $\times 1$ .

Imagen tomada de Wikipedia. Referencia: By w:user:snickerdo - come from en.wikipedia, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=923562>

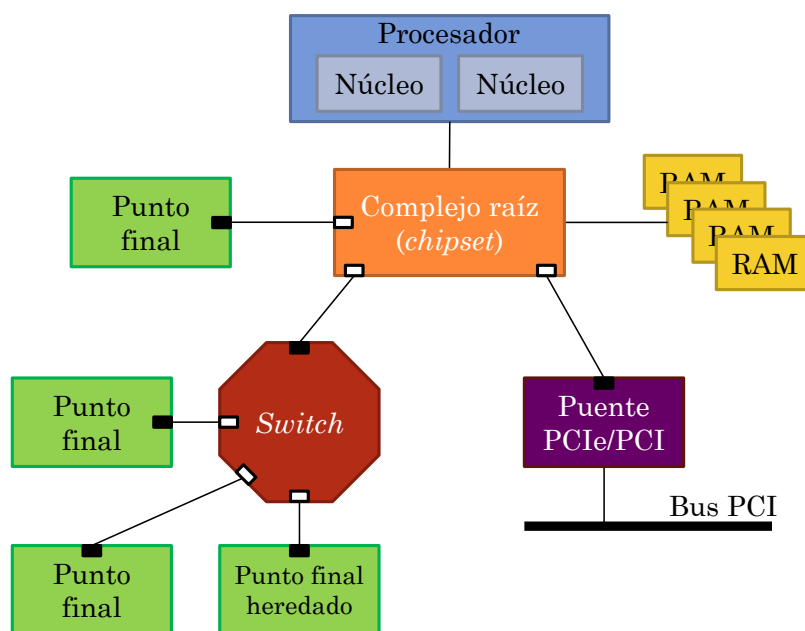
## BUS PCI EXPRESS



35

- PCIe es una arquitectura punto-a-punto con una comunicación serie de datos agrupados en paquetes y enviados a través de carriles bidireccionales (*lanes*).
  - Un puerto PCI puede tener 1, 4, 8, 16 o 32 carriles.
  - La transferencia en cada dirección en un carril se produce mediante señalización diferencial sobre un par de cables.
    - En uno se envía una señal eléctrica (D+).
    - En el otro se envía la misma señal eléctrica, pero de amplitud opuesta (D-).
  - Para codificar el bit enviado, lo que importa es la diferencia de potencial entre los cables, es decir,  $(D+) - (D-)$ .
    - Si la diferencia de potencial es positiva, se interpreta como el bit 1.
      - La máxima diferencia pico a pico ( $V_{Diffp}$ ) puede estar entre 0,8 y 1,2 V.
    - Si la diferencia es negativa, es un bit 0.
  - Ambas señales oscilan alrededor de un potencial común ( $V_{cm}$ ), que puede establecerse en un valor intermedio entre 0 V y 3,6 V.
- Cuando dos dispositivos A y B se comunican entre sí, se dice que existe entre ellos un enlace (*link*).
  - La comunicación es full dúplex, es decir, es bidireccional y simultánea en los dos sentidos.
  - En cada enlace, se dispone de 1 o más carriles (*lanes*) de transmisión serie.
  - Cada carril transmite dos señales, una por sentido.
  - Cada señal consiste en los dos cables trenzados antes mencionados.

## BUS PCI EXPRESS

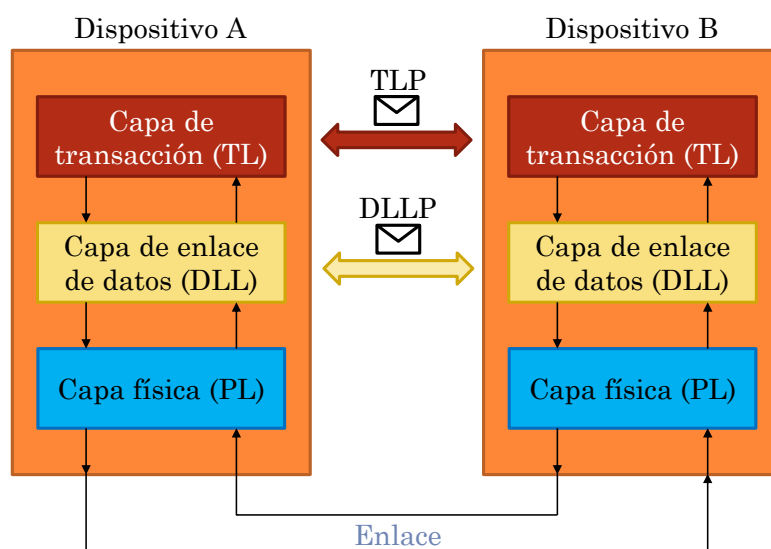


36

- Esta figura muestra la típica topología de uso de PCIe.
  - Se trata de una estructura en forma de árbol, donde el procesador (con sus núcleos o *cores*) se encuentra en la cúspide.
  - El dispositivo complejo raíz (*root complex*), también llamado *chipset* o *host bridge*, conecta el procesador y el sistema de memoria a uno o varios dispositivos de tipo *switch*, de los que a su vez cuelgan los diferentes dispositivos PCIe.
    - El complejo raíz actúa como un dispositivo de almacenamiento temporal (*buffer*) para afrontar la diferencia de velocidades de transmisión de datos entre los controladores de E/S, la memoria y el procesador.
    - También traduce entre los formatos de transacción PCIe y el procesador y las señales de memoria y los requisitos de control.
    - Típicamente el complejo raíz dispone de múltiples puertos PCIe, algunos de los cuales se conectan directamente a un dispositivo PCIe o bien se conectan a un *switch* que gestiona múltiples flujos PCIe.
  - El *switch* es un nodo intermedio en la estructura de árbol que cuelga del complejo raíz (o de otro *switch*) y que sirve para gestionar múltiples flujos PCIe. De él cuelgan a su vez diversos tipos de nodos hoja:
    - Punto final (*endpoint*):
      - Un dispositivo o controlador de E/S que implementa PCIe, tales como una tarjeta de Ethernet Gigabit, un controlador gráfico o de vídeo, una interfaz de disco o un controlador de comunicaciones.
    - Punto final heredado (*legacy endpoint*):
      - Este nodo está previsto para diseños existentes que han sido migrados a PCIe y que permite comportamientos heredados o de legado (antiguos), como el uso del espacio de E/S y transacciones bloqueadas.
    - Puente PCIe/PCI:
      - Permite conectar antiguos dispositivos PCI a los sistemas basados en PCIe.
  - Existen dos tipos de puertos:
    - De bajada (*downstream port*).
      - De ellos cuelga otro componente de la estructura de árbol.
      - Apuntan, por tanto, hacia abajo.
      - En la figura dibujados en color blanco.
    - De subida (*upstream port*).
      - Conectan un componente con el que está inmediatamente más arriba en la estructura de árbol.
      - Apuntan, por tanto, hacia arriba.
      - En la figura dibujados en color negro.

Imagen inspirada en el libro William Stallings, Organización y Arquitectura de Computadoras, Pearson Educación de México, 1.ª edición en español y 11.ª en inglés, 2024.

## BUS PCI EXPRESS



37

TLP = Paquete de la capa de transacción // DLLP = Paquete de la capa de enlace de datos

- Las interacciones PCIe se definen mediante una arquitectura de protocolo basada en capas:
  - Física (PL, *Physical Layer*):
    - Consiste en los cables reales que portan las señales, así como la circuitería y la lógica para soportar características auxiliares requeridas en la transmisión y la recepción de los ceros y los unos.
  - Enlace de datos (DLL, *Data Link Layer*):
    - Es responsable de la transmisión confiable y el control del flujo.
    - Se encarga de reenviar paquetes que han llegado corruptos a su destino gracias a un sistema de detección de errores.
    - Para ello, esta capa se comunica con la capa homónima del dispositivo destino mediante paquetes DLLP (*Data Link Layer Packets*), cuyo formato comentaremos en otra diapositiva.
  - Transacción (TL, *Transaction Layer*):
    - Es la capa superior de la arquitectura, situada justo debajo del software que encarga las peticiones de lectura y escritura.
    - Es la encargada de agrupar los datos de la transferencia en paquetes TLP (*Transaction Layer Packets*), junto con información de enrutamiento (dispositivo destino), tipo de operación, etc.
- Proceso de envío de paquetes TLP desde A hasta B:
  - El dispositivo A recibe una orden del software por encima de él para que envíe información al dispositivo B.
    - Crea entonces un paquete TLP con los datos que se van a enviar.
    - Le añade una cabecera previa con información de enrutamiento y un cierre final con información de detección de errores.
  - Este paquete pasa ahora por la capa de enlace de datos de A:
    - Le añade una cabecera y un cierre (a modo de bocadillo) con información necesaria para el correcto envío.
  - Y, a su vez, pasa por la capa física de A, que también añade una cabecera y un cierre.
    - Esta es la capa que realmente envía el paquete a través del enlace entre ambos dispositivos.
  - Cuando el paquete llega a B, se realiza el mismo proceso, pero en sentido inverso:
    - La capa física de B quita la envoltura física que puso A.
    - La capa de enlace de datos de B quita a su vez la envoltura de enlace de datos de A y comprueba que no ha habido ningún bit corrupto en la transmisión.
    - Por último, la capa de transacción de B quita la envoltura de transacción de A y se queda con los datos (también realiza una comprobación de que no ha habido errores).
- De manera complementaria, las capas de enlace de datos de A y de B se envían entre sí paquetes DLLP para informar sobre el éxito (o fracaso) del envío del paquete TLP:
  - La capa de enlace de datos de B crea el paquete con reconocimiento del éxito (ACK) o del fracaso (NAK) de la transmisión..
  - La capa física de B crea una cabecera y un cierre (a modo de bocadillo).
  - El paquete DLLP se envía por el enlace entre B y A.
  - Al llegar a A, su capa física quita la envoltura física que puso B.
  - Y, por último, la capa de enlace de datos A quita la envoltura de enlace de datos que puso B.

Imagen inspirada en el libro William Stallings, Organización y Arquitectura de Computadoras, Pearson Educación de México, 1.ª edición en español y 11.ª en inglés, 2024.

## BUS PCI EXPRESS

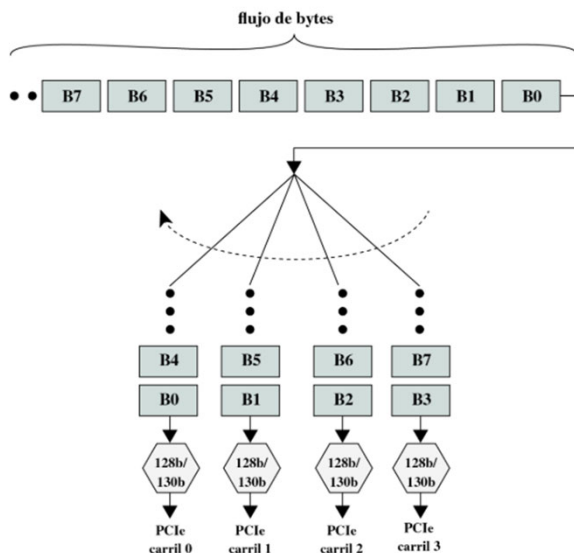


Figura 3.23 Distribución PCIe multicarril.

38

- En lo que sigue, explicaremos la especificación PCIe 3.0 de finales de 2010:
  - Cada puerto PCIe consta de un número de carriles (*lanes*) bidireccionales.
  - En la figura se muestra un ejemplo de PCIe con 4 carriles (sería similar para más carriles).
  - Necesitamos enviar un paquete de datos desde un dispositivo emisor hasta un destino (ver más adelante el formato de los paquetes).
  - Cada byte del paquete se va distribuyendo a cada uno de los 4 carriles usando una técnica simple de rotación o *round-robin* (igual a como se reparten las cartas en los juegos de naipes).
  - En cada carril se van almacenando los bytes recibidos hasta que se crea un bloque de 16 bytes (128 bits).
  - El ancho de banda se define en transferencias de 1 bit por segundo ( $1 \text{ T/s} = 1 \text{ b/s}$ ).
  - Mediante la denominada codificación 128b/130b, el bloque de 128 bits se transforma en una palabra de código única de 130 bits.
    - Es decir, de cada 130 bits enviados, solo 128 son bits de contenido útil.
    - De esta forma, la velocidad de datos efectiva para un carril individual se reduce en un factor 128/130.
    - En la versión anterior de PCIe (versión 2) el ancho de banda para 1 carril era de 500 MB/s (bytes útiles).
      - En este caso, la codificación que se usaba era de 8b/10b.
        - Por cada 8 bits datos, se enviaban 10 bits en total.
        - Esto equivalía a una velocidad de 5 GT/s (gigatransferencias por segundo), ya que  $(500 \text{ MB/s}) \cdot (8 \text{ b/B}) \cdot (10/8) = 5000 \text{ Mb/s} = 5 \text{ Gb/s} = 5 \text{ GT/s}$ .
    - En la PCIe versión 3 se duplicó el ancho de banda respecto de la versión anterior (el doble debería ser 1000 MB/s).
      - Sin embargo, con la codificación 128b/130b, por cada 130 bits enviados, solo 128 bits eran de datos útiles.
        - Por tanto, el verdadero ancho de banda es  $(1000 \text{ MB/s}) \cdot (128/130) = 985 \text{ MB/s}$ .
        - Esto equivale a 8 GT/s.
    - En PCIe versión 6 se ha mejorado el ancho de banda respecto de la versión 3 en un factor 8 ( $=2^3$ ), lo que supone 7,88 GB/s.
      - Esto equivale a 64 GT/s.
      - Si en vez de 1 carril tenemos 16, esto equivale a 126 GB/s.
  - Para entender el trasfondo de la codificación 128b/130b, hay que fijarse que PCIe no usa su línea de reloj para sincronizar el flujo de bits.
    - Esto es, la línea de reloj no se utiliza para determinar el comienzo y el final de cada bit que llega, sino que es usada para otros propósitos de señalización.
    - Sin embargo, es necesario que el receptor esté sincronizado con el transmisor, para que el receptor sepa cuándo empieza y cuándo termina cada bit.
    - Si hay alguna deriva o desajuste entre los relojes usados para la transmisión y recepción de los bits, se pueden producir errores.
    - Para compensar la posibilidad de esta deriva, el PCIe confía en que el receptor se sincronice con el transmisor basándose en la señal transmitida.
    - La sincronización del receptor se consigue buscando transiciones de datos (0->1 o 1->0) y sincronizando su reloj con la transición.
    - Sin embargo, con una cadena larga de ceros o unos con señalización diferencial, la salida es un voltaje constante que dura un largo periodo del tiempo.
    - Bajo estas circunstancias, cualquier deriva entre los relojes del transmisor y del receptor resultará en pérdidas de sincronización entre ambos.

Imagen tomada del libro William Stallings, Organización y Arquitectura de Computadoras, Pearson Educación de México, 1.ª edición en español y 11.ª en inglés, 2024.

## BUS PCI EXPRESS

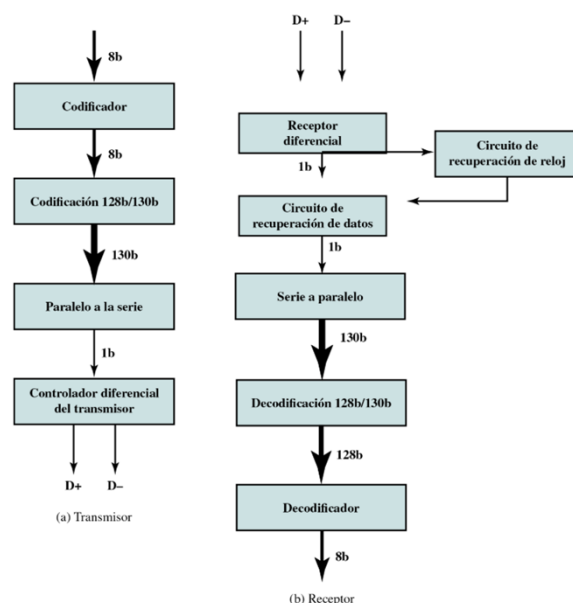
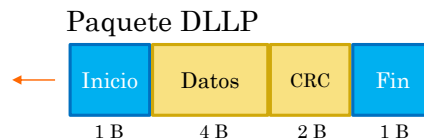
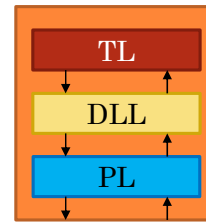


Figura 3.24 Diagramas de bloques de transmisión y recepción PCIe.

- Desde PCIe 3.0 en adelante se usa la técnica de escalamiento o aleatorización (*scrambling*) para superar el problema de las cadenas largas de bits del mismo valor:
  - El escalamiento, que no incrementa el número de bits que hay que transmitir, es una técnica de mapeo que tiende a hacer que los datos parezcan más aleatorios.
    - Aumenta el número de transiciones de tal forma que aparecen al receptor de manera más uniformemente espaciada, lo cual es bueno para la sincronización.
    - También se mejoran otras propiedades de transmisión, como las de tipo espectral, si los datos están repartidos de manera más cercana al caso aleatorio que si fueran constantes o repetitivos.
- Combinado con la técnica de escalamiento, se usa la codificación 128b/130b, en la cual se insertan dos bits adicionales en el flujo de datos para forzar transiciones.
  - En el PCIe 3.0 (y posteriores), cada grupo de 128 bits de entrada se mapea en un bloque de 130 añadiendo una cabecera de sincronización de 2 bits.
  - El valor de la cabecera son los bits 10 para un bloque de datos y 01 para lo que se denomina un bloque de conjunto ordenado, que se refiere a un bloque de información a nivel de la capa de enlace.
- La figura ilustra el uso del escalamiento y la codificación.
  - Los datos que se van a transmitir se proporcionan al “*scrambler*” y se van agrupando de 8 en 8 bits hasta conseguir 128 bits.
  - La salida se proporciona al codificador 128b/130b, que recibe 128 bits y devuelve un bloque de 130 bits.
  - Este bloque pasa entonces a través de un conversor paralelo-serie y se transmite bit a bit mediante señalización diferencial.
  - En el receptor, se sincroniza un reloj a los datos que le llegan para recuperar el flujo de bits.
  - Este pasa a continuación a través de un conversor serie-paralelo para producir un flujo de bloques de 130 bits.
  - Cada bloque se decodifica de 130 a 128 bits para recuperar el patrón de datos con escalamiento, que a su vez es desaleatorizado (“*descrambled*”) para obtener el flujo de bits original.
- Cada transmisión de un bloque de datos sobre el enlace PCI empieza y termina con una secuencia de enmarcado de 8 bits que le da al receptor tiempo para sincronizarse con el flujo de bits entrantes de la capa física.

Imagen tomada del libro William Stallings, Organización y Arquitectura de Computadoras, Pearson Educación de México, 1.ª edición en español y 11.ª en inglés, 2024.

# BUS PCI EXPRESS



40

- Cada uno de estos paquetes TLP se origina en la capa de transacción del dispositivo emisor y termina en la capa de transacción del dispositivo receptor mediante el siguiente proceso:
  - La capa de transacción del emisor crea el paquete con el siguiente formato:
    - Encabezado:
      - 12 o 16 bytes.
      - Describe el tipo de paquete e incluye la información necesaria para que el receptor lo procese adecuadamente, incluyendo cualquier información de enrutamiento necesaria.
    - Datos:
      - De 0 a 4096 bytes.
      - Como puede verse, algunos paquetes no contienen este campo datos.
    - ECRC (*End-to-End Cyclic Redundancy Check*, código de redundancia cíclica de extremo a extremo):
      - Contiene información para detectar si se han producido errores durante el envío en los campos de cabecera y de datos.
  - La capa de enlace de datos añade dos campos más al paquete TLP:
    - Un número de secuencia de 2 bytes (antes de la cabecera).
    - Un código de detección de errores (LCRC, *Link CRC*, código de redundancia cíclica de enlace) de 4 bytes (después del campo ECRC).
  - A su vez, la capa física añade:
    - Un código de inicio (1 byte).
      - Este es el primer byte del paquete que se envía.
      - Su función es dar tiempo al receptor para que se sincronice al flujo de datos que está recibiendo.
    - Y otro que marca el fin del paquete (1 byte).
- Una vez recibido el paquete TLP por parte del receptor:
  - Va eliminando en sentido inverso las envolturas de las capas física, de enlace de datos y de transacción.
  - En este proceso se comparan los datos recibidos con los códigos de detección de errores.
    - Si no se detectan errores:
      - La parte central del paquete contiene los datos, que son válidos, y se entregan al dispositivo receptor.
        - Si este dispositivo receptor es el destino previsto, entonces la capa de transacción procesa el paquete.
        - Si este dispositivo es solo un intermediario, se crea un nuevo paquete TLP con estos datos y se envían al siguiente enlace en el camino hacia el destino.
      - Además, la capa de enlace de datos del receptor envía un paquete DLLP a la capa de enlace de datos del emisor.
        - El mensaje es de tipo ACK (*acknowledge*), que indica que la transmisión del paquete fue correcta.
        - Cuando el dispositivo emisor recibe este mensaje ACK, destruye la copia temporal del paquete TLP que tenía guardada por si acaso.
    - Si se detecta algún bit erróneo:
      - El paquete recibido es borrado, ya que la información no se puede recuperar (se detecta que hay un error, pero no se puede corregir).
      - La capa de enlace de datos del receptor envía un paquete DLLP a la capa de enlace de datos del emisor.
        - Ahora el mensaje es de tipo NAK (*negative acknowledge*), que indica que la transmisión del paquete fue errónea.
        - Cuando el dispositivo emisor recibe este mensaje NAK, vuelve a enviar la copia temporal del paquete TLP que tenía guardada precisamente para estos casos.
- Como puede verse, el formato del paquete DLLP es:
  - Contenido (4 bytes).
  - Código CRC de detección de errores (2 bytes).
  - Tanto a los paquetes TLP como a los DLLP se les añade una envoltura de inicio y cierre por parte de la capa física.

Imagen inspirada en el libro William Stallings, Organización y Arquitectura de Computadoras, Pearson Educación de México, 1.ª edición en español y 11.ª en inglés, 2024.